Lab 4 & 5 – Unit Testing

Objective

1. Understand the purpose of unit testing.
2. Write unit tests using Python's `unittest` framework.
3. Test functions with different types of inputs.
4. Interpret test results and coverage reports.

Description

- Unit testing ensures correctness of individual functions/modules.

Definition

Unit testing is a software testing method where individual units or functions of code are tested independently to ensure they work as expected.

Why Unit Testing is Important

- Detects bugs early in development.
- Improves code quality.
- Makes maintenance easier.
- Helps ensure new changes do not break existing functionality.

The unittest Module

- unittest is a built-in Python module to write and run tests.
- **Key Components:**
    - **Test Case:** A single unit of testing.
    - **Test Suite:** A collection of test cases.
    - **Test Runner:** Executes test cases and reports results.

Testing Tool

- Python `unittest` framework, another tool is jetbrains.

How to set Python Virtual Environment

- **check Python**
  python –version

- **Create a project folder**
  mkdir myproject
  cd myproject

- **Create the virtual environment (venv)**
  python -m venv venv

- **For powershell**
  .\venv\Scripts\Activate.ps1
- **For CMD**
  .\venv\Scripts\activate.bat

- **Select the venv interpreter in VS Code**

1. Press Ctrl+Shift+P (Cmd+Shift+P on mac) → **Python: Select Interpreter**
2. Pick the interpreter that points to your project venv e.g. ./myproject/venv/Scripts/python.exe or venv/bin/python.

- **If it won't work, try this line of code in windows PowerShell (Run as Admin)**

  Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

Task Question

Write unit tests for a calculator program (add, subtract, multiply, divide).

Task Steps

1. Write calculator functions in `calculator.py`.
2. Write test cases in `test_calculator.py`.
3. Run tests and observe results.

Task Code

calculator.py

```
def add(x, y): return x + y
def subtract(x, y): return x - y
def multiply(x, y): return x * y
def divide(x, y):
    if y == 0:
        return "Error"
    return x / y
```

test_calculator.py

```
import unittest
import calculator  # assuming your functions are in calculator.py

class TestCalculator(unittest.TestCase):

    def test_add(self):
        self.assertEqual(calculator.add(2, 3), 5)
        self.assertEqual(calculator.add(-1, 1), 0)
```

```
    def test_subtract(self):
        self.assertEqual(calculator.subtract(5, 3), 2)
        self.assertEqual(calculator.subtract(0, 3), -3)

    def test_multiply(self):
        self.assertEqual(calculator.multiply(2, 3), 6)
        self.assertEqual(calculator.multiply(-1, 3), -3)
        self.assertEqual(calculator.multiply(0, 5), 0)

    def test_divide(self):
        self.assertEqual(calculator.divide(6, 2), 3)
        self.assertEqual(calculator.divide(5, 2), 2.5)
        self.assertEqual(calculator.divide(5, 0), "Error")  # tests the branch
for y==0
```

Output

```
Ran 4 tests in 0.001s
OK
```

Check Coverage

Pip install coverage
python -m unittest test_calculator.py
coverage run -m unittest test_calculator.py
coverage report –m

```
(venv) PS C:\Users\USER\Documents\Python> python -m unittest test_ca
lculator.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
(venv) PS C:\Users\USER\Documents\Python> coverage run -m unittest t
est_calculator.py
....
----------------------------------------------------------------------
--
Ran 4 tests in 0.001s

OK
(venv) PS C:\Users\USER\Documents\Python> coverage report -m
Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
calculator.py           7      0   100%
test_calculator.py     17      0   100%
-------------------------------------------------
TOTAL                  24      0   100%
(venv) PS C:\Users\USER\Documents\Python>
```

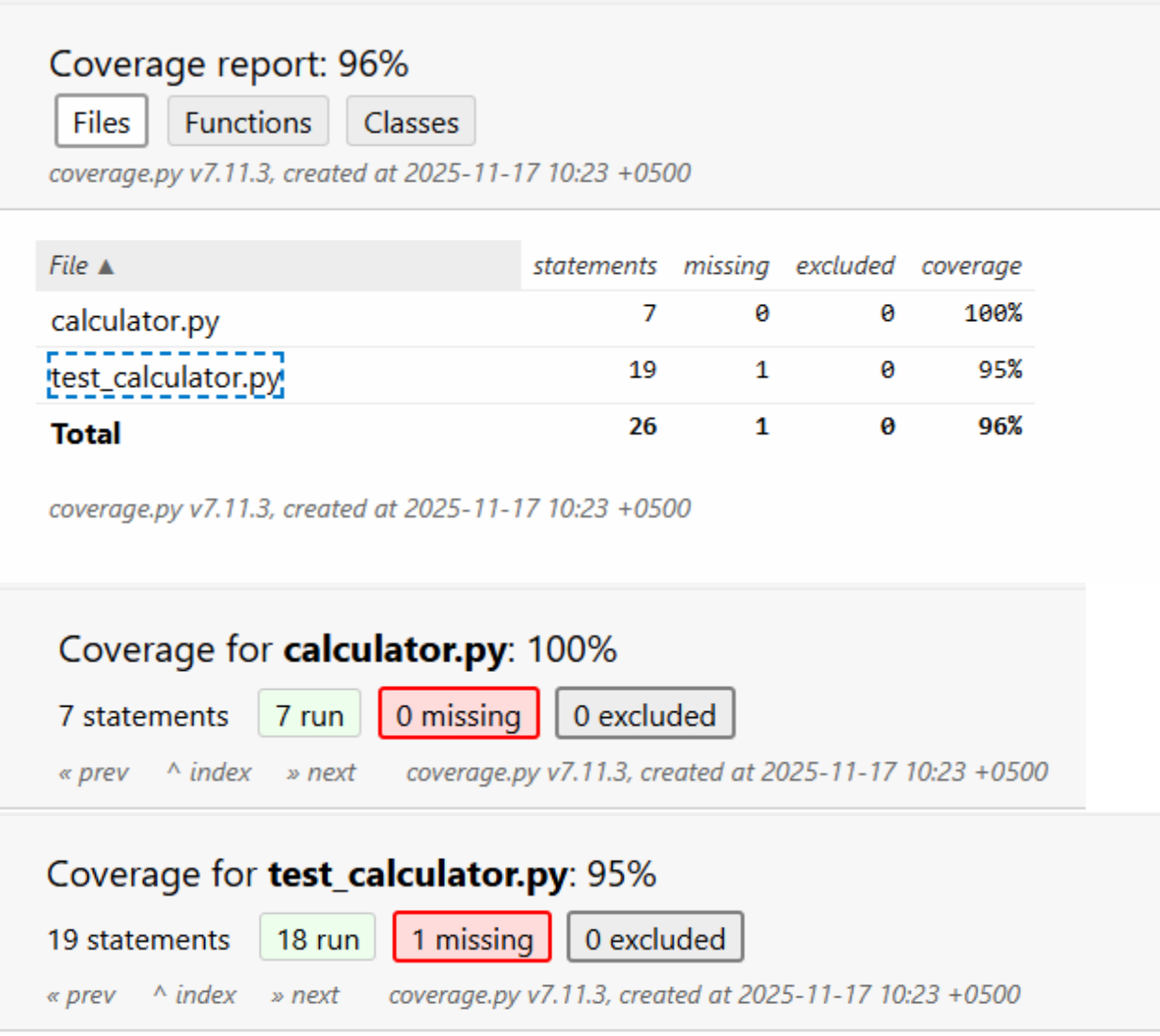
Coverage through HTML:

- coverage html

```
(venv) PS C:\Users\USER\Documents\Python> coverage html
Wrote HTML report to htmlcov\index.html
```

How it works:

- If you want to check where it is no covering line of code,



Result

Unit tests passed successfully.

Do It Yourself

- Implement at least three functions from your project and write unit tests for them. Ensure that your tests achieve 100% code coverage.

Debug

- If tests not running → check file name starts with `test_`.